Learn for Variation: Variationally Guided AAV Trajectory Learning in Differentiable Environments

Xiucheng Wang, Zhenye Chen, Nan Cheng

arXiv:2603.18853v1 [eess.SY] 19 Mar 2026

***Abstract*—Autonomous aerial vehicles (AAVs) empower sixth-generation (6G) Internet-of-Things (IoT) networks through mobility-driven data collection. However, conventional reward-driven reinforcement learning for AAV trajectory planning suffers from severe credit assignment issues and training instability, because sparse scalar rewards fail to capture the long-term and nonlinear effects of sequential movements. To address these challenges, this paper proposes Learn for Variation (L4V), a gradient-informed trajectory learning framework that replaces high-variance scalar reward signals with dense and analytically grounded policy gradients. Particularly, the coupled evolution of AAV kinematics, distance-dependent channel gains, and per-user data-collection progress is first unrolled into an end-to-end differentiable computational graph. Backpropagation through time then serves as a discrete adjoint solver, which propagates exact sensitivities from the cumulative mission objective to every control action and policy parameter. These structured gradients are used to train a deterministic neural policy with temporal smoothness regularization and gradient clipping. Extensive simulations demonstrate that L4V consistently outperforms representative baselines, including a genetic algorithm, DQN, A2C, and DDPG, in mission completion time, average transmission rate, and training cost**

***Index Terms*—Autonomous aerial vehicles, trajectory optimization, variational methods, differentiable simulation, adjoint gradients, neural policy learning.**

I. Introduction

Autonomous aerial vehicles (AAVs) are emerging as a key enabler for sixth-generation (6G) aerial-assisted Internet of Things (IoT) networks [1] by providing on-demand [2], [3] and reconfigurable wireless coverage over large areas [4]. By adjusting their three-dimensional positions, AAVs can reshape the radio link geometry [5], [6], create favorable propagation conditions, and deliver timely connectivity for data collection [7], hotspot offloading [8], and mission-driven sensing in environments where terrestrial infrastructure is unavailable [9]. This capability elevates mobility from a supporting feature to a primary design degree of freedom for network operation [10], [11], since the communication performance experienced by ground users becomes tightly coupled with the AAV flight trajectory over time. Accordingly, this work focuses on communication-aware trajectory planning, where an AAV exploits purposeful motion to improve link quality, expand effective coverage, and accelerate task completion [9], [12], [13]. The central challenge is to coordinate sequential movement decisions with the accumulated communication utility over the entire mission horizon, rather than optimizing instantaneous metrics that may appear locally beneficial but lead to suboptimal system-level efficiency [1], [14].

Despite the apparent simplicity of commanding a moving platform for data collection [15], communication-aware trajectory planning is fundamentally difficult because the achievable data rate is a highly nonlinear [11], [16] and spatially localized function of the AAV position [17]. Small position changes can cause abrupt rate variations, and the service opportunity at each location depends sensitively on the distances to all ground users under heterogeneous task demands [7], [18]. More importantly, practical mission objectives are not determined by instantaneous link quality but by a time-accumulated utility that integrates data-collection progress and service continuity across the entire flight [19]. This long-horizon nature creates a severe credit-assignment difficulty: the value of a movement decision may only become apparent many steps later when the AAV reaches a strategically favorable region [20], [21]. Consequently, the learning signal derived from task outcomes is often sparse or delayed, leading to slow convergence and pronounced training instability for reward-driven methods, especially when deployment configurations change across scenarios. These challenges motivate a trajectory learning paradigm that can provide more informative optimization signals than end-of-horizon scalar rewards and can adapt to diverse operating conditions without extensive retraining [22].

Existing research has pursued two representative directions for communication-aware aerial decision making, yet neither fully resolves these difficulties [10], [20], [23]. Model-based trajectory design methods adopt simplified propagation models and quasi-static service assumptions to obtain tractable formulations and interpretable structural insights [24]. Although effective for revealing design trends and producing principled heuristics, their performance can degrade when missions involve spatially heterogeneous users and time-varying task demands [7]. Deep reinforcement learning (DRL) enables end-to-end decision making with limited reliance on explicit channel models, offering the potential to handle nonlinear dynamics and high-dimensional observations [3]. However, DRL methods typically demand large numbers of environment interactions, exhibit sensitivity to reward shaping, and suffer from training instability that is further amplified by long-horizon credit assignment and sparse returns [14]. A gap therefore persists between structured approaches that are difficult to adapt and

This work was supported by the National Key Research and Development Program of China (2024YFB907500).

Xiucheng Wang and Nan Cheng are with the State Key Laboratory of ISN and School of Telecommunications Engineering, Xidian University, Xi'an 710071, China (e-mail: xcwang_1@stu.xidian.edu.cn; dr.nan.cheng@ieee.org);*(Corresponding author: Nan Cheng.)*.

Zhenye Chen is with the School of Aerospace Science and Technology, Xidian University, Xi'an 710071, China (e-mail: 24149100028@stu.xidian.edu.cn)

flexible approaches whose training cost and stability are insufficient for efficient deployment.

To bridge this gap, this paper develops Learn for Variation (L4V), a gradient-informed trajectory learning framework that injects first-order optimality information from variational principles into neural policy training. The key idea is to express the mission objective as a time-accumulated data-backlog cost and to unroll the entire flight evolution as an end-to-end differentiable computation graph, so that the influence of each movement decision on the final mission outcome can be traced through the underlying system dynamics. Backpropagation through time then serves as a discrete adjoint solver [25], [26], propagating exact sensitivities from the global objective to every control action and policy parameter. The resulting gradients are dense, low-variance, and structurally consistent with the communication mechanism, directly indicating how each local motion choice reshapes future service opportunities and long-term utility. These gradients replace the high-variance scalar reward signals of conventional policy gradient methods, thereby improving convergence speed and training stability. L4V further incorporates temporal smoothness regularization and norm-based gradient clipping to produce physically plausible trajectories and to prevent gradient explosion in long-horizon optimization. The main contributions of this paper are summarized as follows.

1) We formulate communication-aware AAV data collection as a discrete-time optimal control problem in which the mission objective is expressed as a cumulative data-backlog cost over the flight horizon. By deriving first-order necessary conditions via the Pontryagin minimum principle [26], the trajectory learning problem admits principled action-level supervision that goes beyond instantaneous greedy heuristics.
2) We construct an end-to-end differentiable simulation environment that captures the coupled evolution of AAV kinematics, distance-dependent channel gain, and per-user data-collection progress. The entire trajectory is unrolled as a differentiable computation graph, and backpropagation through time implements a discrete adjoint solver that yields exact, low-variance gradients of the global objective with respect to sequential actions and policy parameters.
3) We propose L4V, which integrates the resulting adjoint gradients into deterministic neural policy learning. Temporal smoothness regularization and norm-based gradient clipping are incorporated to improve trajectory regularity, convergence speed, and training stability in long-horizon settings.
4) We conduct systematic simulations against representative baselines spanning evolutionary optimization, value-based reinforcement learning, and actor-critic reinforcement learning, and vary user density, service-area size, unit-distance channel gain, and noise power. Results show that L4V consistently achieves lower mission completion time, higher average transmission rate, and reduced training cost in both iterations and wall-clock time, with the advantages widening under larger problem scales and more challenging channel conditions.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system model and problem formulation. Section IV develops the differentiable sequence optimization method and the associated theoretical analysis. Section V details the L4V framework and its training procedure. Section VI reports simulation results and comparative evaluations. Section VII concludes the paper and discusses future directions.

II. Related Works

The deployment of AAV in wireless networks transforms the static terrestrial infrastructure into a dynamic, three-dimensional ecosystem, introducing a trajectory optimization problem characterized by pathological non-convexity and high-dimensional state spaces. Existing literature has largely bifurcated into three distinct methodological streams: rigorous convex optimization, combinatorial graph theory, and data-driven reinforcement learning.

The dominant framework for offline trajectory planning relies on Convex Optimization, prioritizing theoretical convergence and strict constraint satisfaction. Due to the tight coupling of trajectory variables, transmit power, and user scheduling within the Shannon capacity expression, block coordinate descent (BCD) is widely employed to decompose the master problem into tractable sub-problems [27]. To address the non-convexity arising from the Euclidean distance term in the channel model, successive convex approximation (SCA) is utilized to construct convex surrogates via first-order Taylor expansions, enabling the application of efficient interior-point solvers [28]. This BCD-SCA paradigm is particularly effective for maximizing max-min fairness, where the trajectory is optimized to serve bottleneck users by reformulating the objective via epigraph variables [29]. However, the computational intensity of solving quadratically constrained quadratic programs (QCQP) at each iteration renders this approach unsuitable for real-time applications [30]. Furthermore, these methods depend heavily on smooth, analytical channel models, failing to capture discrete environmental obstacles or complex multipath effects without oversimplified convex exclusion zones.

To mitigate the computational burden of continuous optimization, a subset of literature discretizes the AAV mission into a sequence of logical decisions, leveraging Graph Theory. By abstracting hover locations as nodes, which are often derived via K-Means clustering or geometric intersection of communication disks [31], the trajectory problem is reduced to variants of the traveling salesman problem (TSP) [32] or vehicle routing problem (VRP) [33]. These methods excel in latency minimization and multi-UAV routing tasks, utilizing algorithms like Min-Max Cycle Cover to balance flight loads across a swarm. In obstructed urban environments, grid-based search algorithms such as A* and visibility graphs are employed to ensure collision-free routing. Despite their efficacy in global routing, graph-based methods inherently abstract away physical layer dynamics [34]. They typically assume constant velocities and simplified connectivity models, ignoring the potential throughput gains achievable through

velocity adaptation in fading channels. Consequently, graph solutions are often relegated to initialization roles for more granular solvers.

The exigency for autonomy in dynamic, unknown environments has driven the adoption of deep reinforcement learning (DRL) [35], framing trajectory design as a Markov decision process (MDP) [3]. Algorithms such as deep deterministic policy gradient (DDPG) [36] and twin delayed DDPG (TD3) [37] enable UAVs to learn continuous control policies through trial-and-error, handling complex objectives that blend throughput, energy efficiency, and safety. TD3, in particular, mitigates the overestimation bias inherent in DDPG by employing clipped double Q-learning, yielding smoother and safer trajectories in obstacle-rich domains. For multi-agent scenarios, proximal policy optimization (PPO) is favored for its stability and resistance to catastrophic forgetting [38]. Nevertheless, standard DRL suffers from severe sample inefficiency and the sparse reward problem, where agents fail to converge without extensive reward shaping or curriculum learning. More critically, these model-free agents treat the wireless environment as a black box, ignoring the known physics of electromagnetic propagation, which leads to a significant performance gap when deploying policies trained on statistical models to physical reality.

This context underscores the necessity of L4V. Unlike pure RL which ignores channel physics, or pure optimization which cannot adapt online, L4V is designed to bridge the gap between model-based rigor and data-driven flexibility. By integrating the physical constraints of the wireless medium directly into the learning representation, L4V addresses the limitations of high computational cost and the Sim-to-Real discrepancy, enabling trajectory optimization that is both physically compliant and operationally agile.

III. System Model and Problem Formulation

We consider a wireless communication system where a single AAV is dispatched to collect data from a set of N ground users, $\mathcal{N} = \{1, ..., N\}$, located at fixed coordinates $\mathbf{w}_i = [x_i, y_i]^T$. Each user i must transmit a data packet of size D_i to the UAV. The mission is discretized into T time slots, each with a small duration τ. The AAV flies at a constant altitude H with an adjustable speed $v[t] \in [0, v_{\max}]$. Its trajectory is controlled by a sequence of control inputs $\{\mathbf{u}[t]\}_{t=1}^{T}$, where each $\mathbf{u}[t] = [v[t], \theta[t]]^T$ specifies the flight speed and heading angle at time slot t. The horizontal position $\mathbf{q}[t] = [x[t], y[t]]^T$ is updated according to $\mathbf{q}[t+1] = \mathbf{q}[t] + v[t]\tau[\cos(\theta[t]), \sin(\theta[t])]^T$. The AAV serves all N users simultaneously using an orthogonal frequency division multiple access (OFDMA) scheme. The total system bandwidth B is divided equally among all users, with each user being allocated a dedicated sub-band of width B/N. The communication channel is assumed to be dominated by the Line-of-Sight (LoS) link. The achievable data rate from user i to the AAV in time slot t is therefore given by

$$R_i[t] = \frac{B}{N} \log_2 \left(1 + \frac{p}{(\|\mathbf{q}[t] - \mathbf{w}_i\|^2 + H^2)\, \sigma^2} \right) \tag{1}$$

where $\|\cdot\|$ is the Euclidean norm, p is the transmission power, and σ^2 is the noise power. The objective is to design the UAV's trajectory by optimizing the sequence of control inputs $\{\mathbf{u}[t]\}$ to minimize the average task completion time for all users. Let T_i be the mission time required for user i to complete its data transmission. The optimization problem is formulated as follows

$$\text{(P1)}: \quad \min_{\{\mathbf{u}[t]\}} \quad \frac{1}{N} \sum_{i=1}^{N} T_i \tag{2a}$$

$$\text{s.t.} \quad \sum_{t=1}^{T_i/\tau} R_i[t] \cdot \tau \geq D_i, \quad \forall i \in \mathcal{N} \tag{2b}$$

$$0 \leq v[t] \leq v_{\max}, \quad \forall t \tag{2c}$$

$$\theta[t] \in [0, 2\pi), \quad \forall t \tag{2d}$$

where the trajectory $\mathbf{q}[t]$ embedded in the rate expression $R_i[t]$ is implicitly determined by the sequence of control inputs $\{\mathbf{u}[t]\}$. Problem (P1) is a challenging non-convex optimization problem, as the UAV's trajectory decisions are intricately coupled across all users and time slots through a non-linear objective function. This complexity motivates our development of an efficient learning-based solution.

IV. Differential Sequence Optimization Based Method

A. Theoretical Analysis

The optimization problem (P1) is formulated over a long time horizon, making it computationally challenging to solve directly. To address this, we propose a differentiable trajectory optimization framework, as illustrated in Fig. 1. This architecture unrolls the AAV interaction over time, enabling the computation of exact gradients via the adjoint method. To gain insight into the structure of an optimal policy, we first analyze a localized version of the problem.

The global objective of minimizing the average completion time is equivalent to maximizing the data collection efficiency over the entire mission. At any given time slot t, the most effective immediate action is one that maximizes the progress for users who still have data to transmit. Let us define the set of "active" users at the beginning of slot t as $\mathcal{A}[t] = \{i \in \mathcal{N} \mid \sum_{k=1}^{t-1} R_i[k] \cdot \tau < D_i\}$. A highly effective greedy policy would be to choose the control input $\mathbf{u}[t]$ that maximizes the aggregate data rate for all currently active users. This reformulates the local objective at each time slot t as

$$\text{(P2)}: \quad \max_{\mathbf{u}[t]} \quad J(\mathbf{u}[t]) = \sum_{i \in \mathcal{A}[t]} R_i[t+1] \tag{3}$$

where $R_i[t+1]$ is the rate achievable at the UAV's next position, $\mathbf{q}[t+1]$, which is a direct function of the chosen control input $\mathbf{u}[t]$. This greedy approach aims to make the most progress possible in every single step, which provides a strong heuristic for minimizing the overall mission time.

While the greedy policy presented in (P2) provides a valuable heuristic, it is inherently myopic. By optimizing only the immediate data rate at each step, it fails to account for the long-term consequences of its actions. For instance, a locally optimal move might position the AAV unfavorably for servicing other, more distant users later in the mission, leading to a globally suboptimal trajectory. To overcome these limitations, we

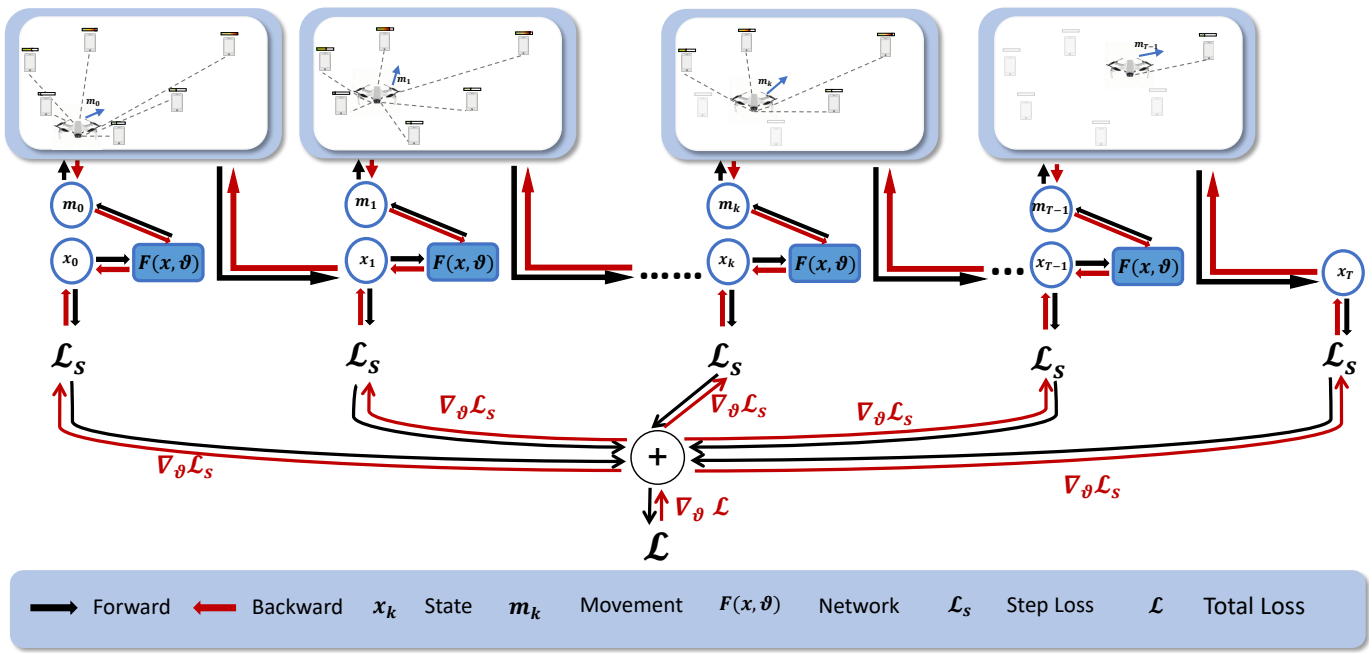


Fig. 1: Illustration of the proposed L4V framework. The diagram illustrates the end-to-end differentiable simulation architecture unrolled over time steps. The forward pass (black arrows) generates the AAV trajectory and accumulates the task loss $\mathcal{L}$ based on the system dynamics $F(x, \theta)$. The backward pass (red arrows) executes BPTT, which serves as a discrete adjoint solver to propagate analytical gradients $\nabla_\theta \mathcal{L}$ directly from the global objective to the policy parameters, ensuring physically grounded learning signals.

reframe the AAV trajectory optimization task from a variational perspective, which shifts the focus from finding a sequence of locally optimal actions to determining an entire trajectory that minimizes a global cost functional. This approach allows us to leverage the powerful tools of optimal control theory. To formalize this, we first define the complete system state at time step t as $\mathbf{x}[t]$, which must encapsulate all information necessary for the system's evolution. This includes both the UAV's physical position $\mathbf{q}[t] \in \mathbb{R}^2$ and the vector of remaining data to be collected, $\mathbf{d}[t] = [d_1[t], \ldots, d_N[t]]^T \in \mathbb{R}^N$. The composite state is therefore $\mathbf{x}[t] = [\mathbf{q}[t]^T, \mathbf{d}[t]^T]^T \in \mathbb{R}^{2+N}$. The trajectory is steered by a two-dimensional control input $\mathbf{u}[t] = [v[t], \theta[t]]^T \in \mathbb{R}^2$, where $v[t] \in [0, v_{\max}]$ is the flight speed and $\theta[t] \in [0, 2\pi)$ is the heading angle. The evolution of this state is governed by the system dynamics, a function $\mathbf{f}$ such that $\mathbf{x}[t+1] = \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t])$. This function comprises the kinematic update $\mathbf{q}[t+1] = \mathbf{q}[t] + v[t]\tau[\cos(\theta[t]), \sin(\theta[t])]^T$ and the task update $d_i[t+1] = \max(0, d_i[t] - R_i(\mathbf{q}[t]) \cdot \tau)$ for each user i. Central to the variational approach is the definition of a cost functional J that aggregates costs over the entire mission horizon T. We define an instantaneous cost $L(\mathbf{x}[t], \mathbf{u}[t]) = \sum_{i=1}^{N} d_i[t]$, representing the total service backlog at that time. The global objective is to minimize the total cost functional, which is the sum of these instantaneous costs:

$$
\begin{aligned}
(\text{P3}): \quad & \min_{\{\mathbf{u}[t]\}_{t=0}^{T-1}} \quad J = \sum_{t=0}^{T-1} L(\mathbf{x}[t], \mathbf{u}[t]) = \sum_{t=0}^{T-1} \sum_{i=1}^{N} d_i[t] \\
\text{s.t.} \quad & \mathbf{x}[t+1] = \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t]), \quad \forall t \in [0, T-1] \\
& 0 \le v[t] \le v_{\max}, \quad \theta[t] \in [0, 2\pi), \quad \forall t \\
& \mathbf{x} = \mathbf{x}_0 = [\mathbf{q}_0^T, D_1, \ldots, D_N]^T
\end{aligned}
\tag{4}
$$

Problem (P3) is a functional minimization task subject to nonlinear dynamics. This formulation transforms the problem into one of finding an optimal control sequence that minimizes the cumulative cost, providing a more principled and globally aware foundation for developing a solution than the greedy approach. Problem (P3) serves as a surrogate objective for the original problem (P1) rather than a strict mathematical equivalence. Minimizing the cumulative backlog $\sum_t \sum_i d_i[t]$ provides a dense, per-step learning signal that encourages the AAV to reduce every user's remaining data as early as possible, but it does not, by itself, guarantee that all backlogs reach zero within the finite horizon T. The connection between the two formulations can be understood as follows. If the horizon T is chosen sufficiently large such that the optimal policy under (P3) drives all backlogs to zero by some time $T^* < T$ (i.e., $d_i[T^*] = 0$ for all i), then every time step beyond T^* contributes zero additional cost, and the policy that minimizes (P3) also minimizes the average completion time in (P1). In our implementation, this condition is enforced in practice: as described in Section VI-A, a trial terminates when the total remaining task falls below $10^{-3}K$, ensuring that near-complete data collection is achieved. Consequently, (P3) effectively aligns with (P1) under the operating conditions of our experiments. We adopt the surrogate formulation because it yields a smooth, differentiable objective that is amenable to gradient-based optimization, whereas (P1) involves the non-

smooth operator $T_i = \min\{t : \sum_{k=1}^{t} R_i[k]\tau \geq D_i\}$, which complicates direct differentiation.

Solving the optimal control problem (P3) requires finding a control sequence that satisfies the first-order necessary conditions for optimality. Classical methods from the calculus of variations, particularly the Pontryagin Minimum Principle (PMP) for discrete-time systems, provide a rigorous framework for deriving these conditions through the introduction of co-state (or adjoint) variables. This theoretical foundation provides a powerful insight for designing a more effective learning-based approach. The PMP framework begins by defining the Hamiltonian function, $\mathcal{H}$, at each time step t:

$$\mathcal{H}(\mathbf{x}[t], \mathbf{u}[t], \boldsymbol{\lambda}[t+1]) = L(\mathbf{x}[t], \mathbf{u}[t]) + \boldsymbol{\lambda}[t+1]^T \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t]) \tag{5}$$

where $\boldsymbol{\lambda}[t] \in \mathbb{R}^{2+N}$ is the co-state (or adjoint) vector. The co-state vector is not merely a mathematical convenience; it has a profound physical interpretation as the sensitivity of the total cost functional J to an infinitesimal perturbation in the state $\mathbf{x}[t]$. That is, $\boldsymbol{\lambda}[t] = \nabla_{\mathbf{x}[t]} J$. These co-states are governed by their own backward dynamics, known as the adjoint equations:

$$\boldsymbol{\lambda}[t] = \nabla_{\mathbf{x}[t]} \mathcal{H}(\mathbf{x}[t], \mathbf{u}[t], \boldsymbol{\lambda}[t+1]) \tag{6}$$

with the terminal condition $\boldsymbol{\lambda}[T] = \nabla_{\mathbf{x}[T]} L(\mathbf{x}[T])$. According to the PMP, a necessary condition for an optimal control sequence $\{\mathbf{u}^*[t]\}$ is that it must minimize the Hamiltonian at every step t along the optimal state trajectory $\{\mathbf{x}^*[t]\}$. For the interior of the control domain where the box constraints $0 \leq v[t] \leq v_{\max}$ are inactive, the stationarity condition implies:

$$\nabla_{\mathbf{u}[t]} \mathcal{H}(\mathbf{x}^*[t], \mathbf{u}^*[t], \boldsymbol{\lambda}^*[t+1]) = \mathbf{0} \tag{7}$$

When the speed constraint is active (i.e., $v^*[t] = 0$ or $v^*[t] = v_{\max}$), the optimality condition for the speed component is replaced by the corresponding complementarity condition, while the stationarity condition for the heading angle $\theta[t]$ continues to hold in the unconstrained sense. This equation provides a direct, analytical gradient that indicates how a change in the control $\mathbf{u}[t]$ affects the global cost J. This gradient, $\nabla_{\mathbf{u}[t]} J = \nabla_{\mathbf{u}[t]} \mathcal{H}$, is precisely the structured, physically-meaningful supervision signal that is often missing in other learning paradigms. This variational insight offers a profound advantage when contrasted with conventional Reinforcement Learning (RL) methods. Standard RL approaches typically learn from scalar rewards, which are local in time and often sparse. To estimate the policy gradient, they rely on stochastic sampling and score-function estimators (e.g., REINFORCE), which are known to suffer from high variance, especially over long horizons. This leads to unstable training and low sample efficiency. The gradient signal from RL, $\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\mathbf{u}|\mathbf{x}) \cdot \hat{A}$, lacks the structural information embedded in the system dynamics. In contrast, the adjoint method provides a deterministic, low-variance vector gradient for each action that is directly derived from the system's underlying physics and the global objective.

Inspired by this, we design a deterministic neural policy, $\pi_{\boldsymbol{\theta}}$, that directly parameterizes the control law as a function of the current state:

$$\mathbf{u}[t] = \pi_{\boldsymbol{\theta}}(\mathbf{x}[t]) \tag{8}$$

Here, $\boldsymbol{\theta}$ represents the trainable parameters (weights and biases) of the neural network. The network outputs the two-dimensional control vector $\mathbf{u}[t] = [v[t], \theta[t]]^T$, where the speed component is passed through a sigmoid activation scaled by $v_{\max}$ to enforce the box constraint $v[t] \in [0, v_{\max}]$, and the heading angle is produced without constraint. The objective is now to find the optimal parameters $\boldsymbol{\theta}^*$ that minimize the cost functional J. The key idea is to use the variational gradients, derived from the adjoint method, to update these parameters. Using the chain rule, the gradient of the total cost J with respect to the network parameters $\boldsymbol{\theta}$ can be expressed as the sum of contributions from each time step:

$$\frac{dJ}{d\boldsymbol{\theta}} = \sum_{t=0}^{T-1} \frac{\partial J}{\partial \mathbf{u}[t]} \frac{\partial \mathbf{u}[t]}{\partial \boldsymbol{\theta}} \tag{9}$$

From the PMP, we have the analytical expression for the gradient of the cost with respect to the action, $\frac{\partial J}{\partial \mathbf{u}[t]} = \nabla_{\mathbf{u}[t]} \mathcal{H}[t]$. The second term, $\frac{\partial \mathbf{u}[t]}{\partial \boldsymbol{\theta}} = \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(\mathbf{x}[t])$, is the Jacobian of the neural network output with respect to its parameters, which can be computed efficiently via standard backpropagation. This leads to the final gradient for updating the policy network:

$$\nabla_{\boldsymbol{\theta}} J = \sum_{t=0}^{T-1} \left(\nabla_{\mathbf{u}[t]} \mathcal{H}[t]\right)^T \left(\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(\mathbf{x}[t])\right) \tag{10}$$

This formulation yields a low-variance, physically meaningful learning signal that propagates information about the global objective directly to the parameters of the neural policy. By replacing noisy, scalar reward signals with structured adjoint gradients, we can achieve significantly more stable and efficient convergence toward a locally optimal control policy.

B. Environment with Differentiable Transition

The theoretical framework of the PMP provides the exact form of the gradients required for optimal control. However, to leverage these gradients for training a neural policy $\pi_{\boldsymbol{\theta}}$, we must establish a practical mechanism for their computation. We achieve this by constructing the entire simulation environment as a single, end-to-end differentiable computational graph. This approach allows us to exploit the power of modern automatic differentiation (AD) frameworks. The key insight is that reverse-mode AD, when applied to a time-unrolled trajectory, is a direct computational implementation of the discrete adjoint method, thus enabling the calculation of the exact variational gradients. To construct this graph, every component of the simulation—from the system dynamics to the cost calculation—must be implemented using differentiable operations. The entire mission trajectory, from the initial state $\mathbf{x}$ to the final cost J, is thereby represented as a deep, sequential function whose parameters are those of the policy network, $\boldsymbol{\theta}$.

The state transition function $\mathbf{x}[t+1] = \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t])$ and the instantaneous cost function $L(\mathbf{x}[t], \mathbf{u}[t])$ must be composed exclusively of operations for which derivatives are known. The kinematic update, $\mathbf{q}[t+1] = \mathbf{q}[t] + v[t]\tau[\cos(\theta[t]), \sin(\theta[t])]^T$,

relies on addition, multiplication, and trigonometric functions, which are standard differentiable operations. The task update, $d_i[t+1] = \max(0, d_i[t] - R_i(\mathbf{q}[t]) \cdot \tau)$, involves the 'max' function, which is piecewise differentiable and handled seamlessly by modern AD systems. The rate function $R_i(\mathbf{q}[t])$ itself, as defined in (1), is composed of differentiable norms, additions, and logarithms. Consequently, the entire state transition function $\mathbf{f}$ is differentiable with respect to its inputs $\mathbf{x}[t]$ and $\mathbf{u}[t]$. We can define the local sensitivity of the next state to the current state and control via the Jacobian matrices:

$$\mathbf{A}[t] \triangleq \frac{\partial \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t])}{\partial \mathbf{x}[t]} \in \mathbb{R}^{(2+N)\times(2+N)} \tag{11}$$

$$\mathbf{B}[t] \triangleq \frac{\partial \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t])}{\partial \mathbf{u}[t]} \in \mathbb{R}^{(2+N)\times 2} \tag{12}$$

These matrices contain the linearized dynamics around the current trajectory point. The instantaneous cost, $L(\mathbf{x}[t]) = \sum_{i=1}^{N} d_i[t]$, is a linear sum, which is trivially differentiable. Its partial derivatives are:

$$\frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{x}[t]} = \begin{bmatrix} \mathbf{0}_{2\times 1} \\ \mathbf{1}_{N\times 1} \end{bmatrix}^T, \tag{13}$$

$$\frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{u}[t]} = \mathbf{0}_{1\times 2} \tag{14}$$

The training process involves unrolling the entire trajectory over the horizon T. This creates a deep computational structure where the output of one time step becomes the input to the next, with the policy network $\pi_{\boldsymbol{\theta}}$ being applied at each step. This sequence of operations can be expressed as:

$$\begin{aligned} \mathbf{u} &= \pi_{\boldsymbol{\theta}}(\mathbf{x}) \\ \mathbf{x} &= \mathbf{f}(\mathbf{x}, \mathbf{u}) \\ \mathbf{u} &= \pi_{\boldsymbol{\theta}}(\mathbf{x}) \\ \mathbf{x} &= \mathbf{f}(\mathbf{x}, \mathbf{u}) \\ &\vdots \\ \mathbf{u}[T-1] &= \pi_{\boldsymbol{\theta}}(\mathbf{x}[T-1]) \\ \mathbf{x}[T] &= \mathbf{f}(\mathbf{x}[T-1], \mathbf{u}[T-1]) \end{aligned} \tag{15}$$

The total cost functional J is then computed as the sum over the states generated by this unrolling: $J = \sum_{t=0}^{T-1} L(\mathbf{x}[t])$. This entire process, from $\mathbf{x}$ and $\boldsymbol{\theta}$ to the final scalar value J, constitutes a single, well-defined mathematical function: $J = \mathcal{G}(\mathbf{x}, \boldsymbol{\theta})$. The core of our method is to compute the gradient of this function with respect to $\boldsymbol{\theta}$.

When we apply reverse-mode AD (i.e., backpropagation) to the final cost J, the algorithm automatically computes the gradients by traversing the computational graph backward. This process is mathematically equivalent to solving the discrete adjoint equations. The chain rule, as applied by the AD engine, recursively computes the co-state vectors. Starting with the terminal condition $\boldsymbol{\lambda}[T] = \frac{\partial L(\mathbf{x}[T])}{\partial \mathbf{x}[T]}$, the backpropagation algorithm computes:

$$\begin{aligned} \boldsymbol{\lambda}[t] = \frac{dJ}{d\mathbf{x}[t]} &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{x}[t]} + \frac{dJ}{d\mathbf{x}[t+1]} \frac{\partial \mathbf{x}[t+1]}{\partial \mathbf{x}[t]} \\ &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{x}[t]} + \boldsymbol{\lambda}[t+1]^T \mathbf{A}[t] \end{aligned} \tag{16}$$

This is precisely the update rule for the co-state vector in the discrete-time PMP. The term $\frac{dJ}{d\mathbf{x}[t]}$ computed by the AD engine *is* the co-state $\boldsymbol{\lambda}[t]$, which represents the sensitivity of the total cost to a change in the state at time t.

Meanwhile, the AD engine computes the gradient of the total cost with respect to each control input $\mathbf{u}[t]$:

$$\begin{aligned} \frac{dJ}{d\mathbf{u}[t]} &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{u}[t]} + \frac{dJ}{d\mathbf{x}[t+1]} \frac{\partial \mathbf{x}[t+1]}{\partial \mathbf{u}[t]} \\ &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{u}[t]} + \boldsymbol{\lambda}[t+1]^T \mathbf{B}[t] \end{aligned} \tag{17}$$

This expression is the gradient of the Hamiltonian with respect to the control, $\nabla_{\mathbf{u}[t]} \mathcal{H}[t]$. By constructing a fully differentiable simulation, we effectively compel the backpropagation algorithm to perform the discrete adjoint method. This provides the exact, low-variance, action-level gradients $\frac{dJ}{d\mathbf{u}[t]}$ needed to train the policy network $\pi_{\boldsymbol{\theta}}$ via the chain rule, thereby directly connecting the global objective to the parameter updates in a principled and efficient manner.

V. Variation-Inspired Learning Method

The L4V framework operationalizes the preceding variational principles through a system architecture composed of two core, interconnected modules: a parameterized neural policy and a fully differentiable environment simulator. The first module, the control policy $\pi_{\boldsymbol{\theta}}$, serves as a function approximator for the intractable optimal control law. It defines a deterministic mapping from the current system state to a control action, $\mathbf{u}[t] = \pi_{\boldsymbol{\theta}}(\mathbf{x}[t])$, where the behavior is governed by a set of trainable parameters $\boldsymbol{\theta}$. The second module is the differentiable environment simulator, $\mathcal{S}$, which computationally embodies the system dynamics and cost function. Its defining characteristic is that every internal operation is differentiable, transforming it from a traditional "black-box" simulator into a transparent "white-box" computational graph. This ensures that the state Jacobians, $\frac{\partial \mathbf{f}}{\partial \mathbf{x}[t]}$ and $\frac{\partial \mathbf{f}}{\partial \mathbf{u}[t]}$, are well-defined and computable. The synergy between these modules is central to our approach. During training, the policy and simulator are unrolled in a closed-loop fashion over the time horizon T, creating a deep, sequential computational graph. The state at any given time step is thus a direct result of the initial state and the sequence of actions generated by the policy. This recursive relationship, $\mathbf{x}[t+1] = \mathbf{f}(\mathbf{x}[t], \pi_{\boldsymbol{\theta}}(\mathbf{x}[t]))$, reveals that any state $\mathbf{x}[t]$ is a highly nested, composite function of the initial state $\mathbf{x}$ and the policy parameters: $\mathbf{x}[t] = \Phi_t(\mathbf{x}, \boldsymbol{\theta})$. Consequently, the total cost functional J can be expressed as a single, end-to-end differentiable function of the policy parameters $\boldsymbol{\theta}$:

$$J(\boldsymbol{\theta}) = \sum_{t=0}^{T-1} L(\mathbf{x}[t+1]) = \sum_{t=0}^{T-1} L(\Phi_{t+1}(\mathbf{x}, \boldsymbol{\theta})) \tag{18}$$

This final formulation is paramount, as it transforms the complex variational problem of finding an optimal control function into a more conventional, albeit very deep, optimization problem of finding the parameters $\boldsymbol{\theta}$ that minimize the differentiable loss function J. This allows us to apply powerful, gradient-based optimization algorithms directly.

A. Training Procedure

The training procedure for the L4V framework is deterministic and follows a "simulation as computation" paradigm. For each training iteration, the method performs two distinct phases on the computational graph defined in the previous section: a forward pass to simulate the trajectory and compute the cost, and a backward pass to compute the exact variational gradients with respect to the policy parameters. This phase commences by unrolling the system dynamics over a finite time horizon T using the current policy $\pi_{\boldsymbol{\theta}}$. Starting with an initial state $\mathbf{x}$, the trajectory is generated by recursively applying the policy and the state transition function. For each time step $t \in [0, T-1]$:

$$\mathbf{u}[t] = \pi_{\boldsymbol{\theta}}(\mathbf{x}[t]) \tag{19}$$

$$\mathbf{x}[t+1] = \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t]) \tag{20}$$

This forward pass is executed within an automatic differentiation (AD) framework, which records every computation and its dependencies in a dynamic computational graph. Concurrently, the total cost functional J is accumulated by summing the instantaneous costs incurred at each step:

$$J(\boldsymbol{\theta}) = \sum_{t=0}^{T-1} L(\mathbf{x}[t+1]) \tag{21}$$

Upon completion of the rollout, $J(\boldsymbol{\theta})$ exists as the terminal node of the computational graph, representing the total cost as a differentiable function of the policy parameters $\boldsymbol{\theta}$.

The core of the L4V methodology lies in the implementation of the backward pass. By invoking a reverse-mode AD routine on the scalar cost J, we compute the total derivative $\frac{dJ}{d\boldsymbol{\theta}}$. This operation, commonly known as Backpropagation Through Time (BPTT), implicitly performs the discrete adjoint method, providing an efficient and exact calculation of the required variational gradients. The total gradient is found by applying the chain rule over the entire unrolled graph, which decomposes into a sum over the time steps:

$$\frac{dJ}{d\boldsymbol{\theta}} = \sum_{t=0}^{T-1} \frac{dJ}{d\mathbf{u}[t]} \frac{\partial \mathbf{u}[t]}{\partial \boldsymbol{\theta}} \tag{22}$$

Let us analyze the components of this expression. The term $\frac{\partial \mathbf{u}[t]}{\partial \boldsymbol{\theta}} = \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(\mathbf{x}[t])$ is the Jacobian of the neural policy's output with respect to its parameters, which is readily computed via standard backpropagation within the network module. The critical term is $\frac{dJ}{d\mathbf{u}[t]}$, which represents the total derivative of the global cost with respect to the local control action $\mathbf{u}[t]$. This term, the adjoint gradient, captures the full impact of a perturbation in $\mathbf{u}[t]$ on the entire future trajectory and accumulated cost. The BPTT algorithm computes this term by propagating sensitivities backward in time. First, let us define the co-state vector (the adjoint) at time t as the total derivative of the cost functional with respect to the state $\mathbf{x}[t]$:

$$\boldsymbol{\lambda}[t]^T \triangleq \frac{dJ}{d\mathbf{x}[t]} \tag{23}$$

The AD engine calculates these co-states recursively, starting from the terminal state and moving backward. Applying the chain rule at step t:

$$\begin{aligned} \boldsymbol{\lambda}[t]^T = \frac{dJ}{d\mathbf{x}[t]} &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{x}[t]} + \frac{dJ}{d\mathbf{x}[t+1]} \frac{\partial \mathbf{x}[t+1]}{\partial \mathbf{x}[t]} \\ &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{x}[t]} + \boldsymbol{\lambda}[t+1]^T \frac{\partial \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t])}{\partial \mathbf{x}[t]} \end{aligned} \tag{24}$$

This recursive update is precisely the discrete-time adjoint equation. Once the co-state $\boldsymbol{\lambda}[t+1]$ is known, the AD engine can compute the required action gradient:

$$\begin{aligned} \frac{dJ}{d\mathbf{u}[t]} &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{u}[t]} + \frac{dJ}{d\mathbf{x}[t+1]} \frac{\partial \mathbf{x}[t+1]}{\partial \mathbf{u}[t]} \\ &= \frac{\partial L(\mathbf{x}[t])}{\partial \mathbf{u}[t]} + \boldsymbol{\lambda}[t+1]^T \frac{\partial \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t])}{\partial \mathbf{u}[t]} \end{aligned} \tag{25}$$

This expression is equivalent to the gradient of the Hamiltonian, $\nabla_{\mathbf{u}[t]} \mathcal{H}$, providing a direct, physically-grounded learning signal. By automating this backward pass, the L4V framework computes the exact gradient of the global objective with respect to every parameter in the policy network, enabling efficient optimization via standard gradient-based methods. The entire procedure is detailed in Algorithm 1.

While the core L4V framework provides a principled method for policy optimization, its practical application to long-horizon, dynamically complex problems benefits from techniques that enhance training stability and improve the quality of the learned trajectories. We incorporate two such enhancements: a temporal regularization term to ensure control smoothness and gradient clipping to stabilize the learning process. The primary cost functional $J(\boldsymbol{\theta})$ exclusively rewards task completion, providing no incentive for the policy to generate efficient or physically plausible control sequences. Without additional constraints, the resulting policy may exhibit rapid oscillations in the control signal, where the speed and heading angle change drastically between consecutive time steps. Such behavior is not only energetically inefficient but may also be infeasible for a physical AAV due to inertial and actuator constraints. To address this, we introduce a temporal regularization term, J_{smooth}, to the objective function. Since the control input $\mathbf{u}[t] = [v[t], \theta[t]]^T$ contains a heading angle that is periodic, a standard Euclidean difference would produce spurious large penalties when the angle wraps around (e.g., from 0.1 to $2\pi - 0.1$ radians). We therefore decompose the smoothness penalty into a speed component and an angular component:

$$\begin{aligned} J_{\text{smooth}}(\boldsymbol{\theta}) = \sum_{t=1}^{T-1} [&(v[t] - v[t-1])^2 \\ &+ \alpha(1 - \cos(\theta[t] - \theta[t-1]))] \end{aligned} \tag{26}$$

where the first term penalizes abrupt speed changes using the standard squared difference, and the second term penalizes heading changes using a cosine-based metric that naturally respects the 2π-periodicity of the angle. The coefficient $\alpha > 0$ balances the relative importance of speed and heading smoothness.

The new, regularized total cost functional, $J_{\text{total}}(\boldsymbol{\theta})$, is a weighted sum of the original task-oriented cost and this new

smoothness penalty:

$$J_{\text{total}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \beta J_{\text{smooth}}(\boldsymbol{\theta}) \tag{27}$$

where $\beta > 0$ is a regularization coefficient that controls the trade-off between task performance and control smoothness. Since the regularization term is composed of standard differentiable operations, J_{total} remains fully differentiable with respect to $\boldsymbol{\theta}$. The gradient of this new objective is computed during the backward pass. Due to the linearity of the gradient operator, the new total gradient is the sum of the original gradient and the gradient of the regularization term:

$$\nabla_{\boldsymbol{\theta}} J_{\text{total}} = \nabla_{\boldsymbol{\theta}} J + \beta \nabla_{\boldsymbol{\theta}} J_{\text{smooth}} \tag{28}$$

The gradient of the smoothness term contributes an additional component to the action-level gradients during BPTT. For a given control input $\mathbf{u}[\tau] = [v[\tau], \theta[\tau]]^T$, its partial derivatives with respect to J_{smooth} are:

$$\frac{\partial J_{\text{smooth}}}{\partial v[\tau]} = 2(v[\tau] - v[\tau-1]) - 2(v[\tau+1] - v[\tau]) \tag{29}$$

$$\frac{\partial J_{\text{smooth}}}{\partial \theta[\tau]} = \alpha \sin(\theta[\tau] - \theta[\tau-1]) - \alpha \sin(\theta[\tau+1] - \theta[\tau]) \tag{30}$$

The process of Backpropagation Through Time over a long horizon T is analogous to backpropagation through a very deep neural network with shared weights. A common concern in such settings is the exploding gradient problem, where the norm of the gradient vector grows exponentially due to repeated multiplication by state Jacobians. However, the dynamics of the specific AAV system considered here exhibit a distinctive structure that mitigates this concern. The kinematic update $\mathbf{q}[t+1] = \mathbf{q}[t] + v[t]\tau[\cos(\theta[t]), \sin(\theta[t])]^T$ implies that $\frac{\partial \mathbf{q}[t+1]}{\partial \mathbf{q}[t]} = \mathbf{I}_{2\times 2}$, and the task update $d_i[t+1] = \max(0, d_i[t] - R_i(\mathbf{q}[t]) \cdot \tau)$ yields $\frac{\partial d_i[t+1]}{\partial d_i[t]} \in \{0, 1\}$. Consequently, the state Jacobian $\mathbf{A}[k]$ has eigenvalues bounded by one, and the product $\prod_{k=t}^{T-1} \mathbf{A}[k]$ does not exhibit exponential growth. Nevertheless, when the horizon T is large, the gradients from all time steps accumulate linearly in the co-state recursion, and the cross-coupling terms $\frac{\partial d_i[t+1]}{\partial \mathbf{q}[t]}$ (arising from the position dependence of the data rate) can amplify the gradient norm over many steps. To ensure robust training under varying horizon lengths and problem scales, we employ gradient clipping as a standard engineering safeguard. After computing the full gradient vector $\mathbf{g} = \nabla_{\boldsymbol{\theta}} J_{\text{total}}$ but before applying the optimizer step, we rescale the gradient if its L2-norm $||\mathbf{g}||$ exceeds a predefined threshold $C > 0$. The clipped gradient, $\hat{\mathbf{g}}$, is computed as follows:

$$\hat{\mathbf{g}} = \begin{cases} \mathbf{g} & \text{if } ||\mathbf{g}|| \leq C \\ C\frac{\mathbf{g}}{||\mathbf{g}||} & \text{if } ||\mathbf{g}|| > C \end{cases} \tag{31}$$

This can be expressed more compactly as $\hat{\mathbf{g}} = \min\left(1, \frac{C}{||\mathbf{g}||}\right)\mathbf{g}$. This operation preserves the direction of the gradient vector, ensuring that the update remains in a descent direction, but limits its magnitude. By bounding the size of the parameter updates, gradient clipping prevents excessively large steps in the optimization landscape and ensures stable convergence, particularly in complex scenarios with long time horizons.

B. Convergence Analysis and Stability

The L4V framework exhibits convergence properties that are fundamentally different from and demonstrably more stable than those of traditional reinforcement learning methods. This advantage is predicated upon the nature of the gradients used for policy optimization. We analyze the convergence from three perspectives: gradient variance, the physical grounding of the learning signal, and the nature of the convergence point.

The optimization process in L4V seeks to solve the unconstrained minimization problem $\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ via gradient descent, using the update rule $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_k)$, where η_k is the learning rate at iteration k. The gradient, as derived previously, is given by:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_k) = \sum_{t=0}^{T-1} \left(\nabla_{\mathbf{u}[t]} \mathcal{H}(\mathbf{x}[t], \mathbf{u}[t], \boldsymbol{\lambda}[t+1])\right)^T \left(\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}_k}(\mathbf{x}[t])\right) \tag{32}$$

For a given set of parameters $\boldsymbol{\theta}_k$ and a fixed initial state $\mathbf{x}$, this computation is entirely deterministic. The trajectory $\{\mathbf{x}[t], \mathbf{u}[t]\}$ is uniquely determined by the forward pass, and the co-states $\{\boldsymbol{\lambda}[t]\}$ are uniquely determined by the backward pass. Consequently, the gradient has zero variance:

$$\text{Var}[\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_k)] = 0 \tag{33}$$

This stands in stark contrast to the gradients used in standard stochastic policy gradient (PG) methods, which are formulated as an expectation over trajectories τ sampled from the policy: $\nabla_{\boldsymbol{\theta}} J_{\text{PG}} = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}}[\sum_{t=0}^{T-1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) A^{\pi}(s_t, a_t)]$. This expectation is estimated using a Monte Carlo estimator, $\hat{g}_{\text{PG}}$, from a batch of M sampled trajectories:

$$\hat{g}_{\text{PG}} = \frac{1}{M} \sum_{m=1}^{M} \sum_{t=0}^{T-1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_{t,m}|s_{t,m}) A^{\pi}(s_{t,m}, a_{t,m}) \tag{34}$$

The variance of this estimator, $\text{Var}[\hat{g}_{\text{PG}}]$, is typically non-zero and often substantial, scaling with the complexity of the environment and the horizon length T. The high variance leads to noisy update directions, hindering convergence speed and stability. The zero-variance property of the L4V gradient ensures that each parameter update moves the policy in a true descent direction of the loss landscape, enabling smoother and more monotonic convergence.

Furthermore, the backpropagated gradients in L4V are not arbitrary numerical values but represent the precise physical sensitivity of the global cost to each parameter. The action-level gradient, $\nabla_{\mathbf{u}[t]} \mathcal{H}$, is composed of terms like $\boldsymbol{\lambda}[t+1]^T \frac{\partial \mathbf{f}}{\partial \mathbf{u}[t]}$, which explicitly quantifies how a change in control $\mathbf{u}[t]$ affects the future state (via the Jacobian $\frac{\partial \mathbf{f}}{\partial \mathbf{u}[t]}$) and how that change in state impacts the total future cost (via the co-state $\boldsymbol{\lambda}[t+1]$). This ensures that the learning signal is structurally consistent with the system dynamics, guiding the policy toward physically meaningful and efficient behaviors.

Finally, we analyze the convergence point of the L4V algorithm. The optimization problem is non-convex due to the non-linearities in both the neural network $\pi_{\boldsymbol{\theta}}$ and the system dynamics $\mathbf{f}$. As a gradient-based method, L4V is subject

to the standard convergence guarantees for non-convex optimization. Assuming the cost functional $J(\boldsymbol{\theta})$ is continuously differentiable and its gradient $\nabla_{\boldsymbol{\theta}} J$ is Lipschitz continuous with constant L_g, then for a sufficiently small learning rate (e.g., $\eta_k < 2/L_g$), the gradient descent algorithm is guaranteed to find a stationary point $\boldsymbol{\theta}^*$ where the gradient norm vanishes:

$$\lim_{k\to\infty} ||\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_k)|| = 0 \tag{35}$$

A stationary point satisfies the first-order necessary conditions for optimality. While this does not guarantee global optimality, it ensures convergence to a point that is, at minimum, a local optimum or a saddle point. Crucially, the condition $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^*) = 0$ implies that the underlying control sequence $\{\mathbf{u}^*[t]\}$ generated by the converged policy $\pi_{\boldsymbol{\theta}^*}$ approximately satisfies the Pontryagin Minimum Principle's optimality condition, $\nabla_{\mathbf{u}[t]}\mathcal{H} \approx \mathbf{0}$, for all t. Thus, the policies learned by L4V are, by construction, locally optimal with respect to the first-order necessary conditions derived from the calculus of variations.

C. Comparison with Reinforcement Learning

Algorithm 1 Learn for Variation

Initialize policy network parameters $\boldsymbol{\theta}$, optimizer.
for episode = $1 \to N_{\text{episodes}}$ **do**
  Reset environment to initial state $\mathbf{x}$.
  Initialize total cost $J \leftarrow 0$.

  */*Forward Pass: Differentiable Rollout*/*
  for $t = 0 \to T-1$ **do**
    Compute action: $\mathbf{u}[t] \leftarrow \pi_{\boldsymbol{\theta}}(\mathbf{x}[t])$
    Evolve state: $\mathbf{x}[t+1] \leftarrow \mathbf{f}(\mathbf{x}[t], \mathbf{u}[t])$
    Accumulate cost: $J \leftarrow J + L(\mathbf{x}[t+1])$
  end for

  */*Backward Pass: Adjoint Gradient Computation*/*
  Compute gradient of the total cost with respect to the policy parameters: $\nabla_{\boldsymbol{\theta}} J \leftarrow \frac{dJ}{d\boldsymbol{\theta}}$ via Backpropagation Through Time (BPTT).

  */*Parameter Update*/*
  Update policy parameters using the computed gradient and optimizer: $\boldsymbol{\theta} \leftarrow \text{OptimizerStep}(\boldsymbol{\theta}, \nabla_{\boldsymbol{\theta}} J)$
end for

The L4V framework, while addressing the same class of sequential decision-making problems as Reinforcement Learning, represents a fundamentally different optimization paradigm. Its advantages are most pronounced in problems where the system dynamics, though complex, are known and differentiable. We analyze these advantages in terms of gradient quality, sample efficiency, and the mechanism of credit assignment. The primary distinction lies in the method of gradient computation. Most modern RL algorithms, particularly in the policy gradient family, rely on the score-function estimator, which expresses the gradient of the expected return $J_{\text{RL}}(\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}[\sum_t R_t]$ as an expectation:

$$\nabla_{\boldsymbol{\theta}} J_{\text{RL}}(\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}\left[\left(\sum_{t=0}^{T-1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t)\right)\left(\sum_{t=0}^{T-1} R_t\right)\right] \tag{36}$$

In practice, this expectation is approximated using Monte Carlo sampling over a batch of trajectories. This estimation process is inherently stochastic and is the principal source of the high variance that plagues RL algorithms. The resulting noisy gradient estimates can impede convergence, necessitating large batch sizes, sophisticated variance reduction techniques, e.g., baseline subtraction, advantage estimation, and extensive hyperparameter tuning. In stark contrast, the L4V framework computes the gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ analytically and deterministically via BPTT. For a given policy $\pi_{\boldsymbol{\theta}}$, the gradient is unique, yielding a zero-variance estimator that provides a true descent direction in the parameter space, leading to significantly more stable and monotonic convergence. This difference in gradient quality directly translates to a profound advantage in sample and computational efficiency. A single forward-backward pass in L4V on one trajectory provides the exact gradient of the cost functional. Conversely, RL methods may require thousands or millions of sampled trajectories (environment interactions) to average out the noise and obtain a sufficiently reliable gradient estimate. The computational cost of L4V is dominated by the BPTT pass, whose complexity scales linearly with the horizon length T, whereas the cost of RL is dominated by the vast number of environment rollouts required for learning. Therefore, for problems where the simulation is computationally non-trivial, L4V can converge to a high-performance policy orders of magnitude faster than its RL counterparts. Furthermore, L4V inherently solves the temporal credit assignment problem, a long-standing challenge in RL. The difficulty in RL lies in attributing an outcome to the specific actions that caused it, especially when feedback is delayed. The adjoint method, as implemented by BPTT, provides a direct and precise solution. The co-state vector $\boldsymbol{\lambda}[t] = \nabla_{\mathbf{x}[t]} J$ analytically quantifies the exact sensitivity of the total final cost to the state at time t. This allows the backward pass to precisely calculate the action-level gradient $\nabla_{\mathbf{u}[t]}\mathcal{H}$, which explicitly links the local action $\mathbf{u}[t]$ to its ultimate impact on the global objective J. RL, by contrast, relies on temporal difference learning or discounted rewards to propagate credit information backward through time, a process that can be slow, imprecise, and heavily biased by the choice of discount factor γ. By leveraging the full structure of the system dynamics, L4V replaces this difficult estimation problem with a direct, analytical computation.

VI. Simulation Results

A. Simulation Setup

We evaluate mission-level performance under different propagation and noise conditions by reporting the task completion time, the average data transmission rate over the mission horizon, and the training cost in terms of wall-clock time and training iterations. For each setting, every method is tested over 10 independent trials with randomized user locations and

task demands, and the results are averaged across trials. At the beginning of each trial, K users are uniformly distributed in a square region of side length L, namely $[-L/2,\ L/2]^2$, and each user task demand is independently sampled from $[0.5, 1]$. The AAV is initialized at the region center. The default configuration uses $K = 4$, $L = 10$, unit-distance gain $\eta = 1$, and noise power $\sigma_w^2 = 0.1$. We vary one factor at a time with others fixed at default, including $K \in \{2, 4, 6, 8, 10\}$, $L \in \{5, 10, 15, 20, 25\}$, $\eta \in \{0.5, 1, 1.5, 2.0, 2.5\}$, and $\sigma_w^2 \in \{0.05, 0.1, 0.15, 0.2, 0.25\}$. For L4V, the policy iterator is an MLP that takes the AAV position, user relative positions, and the remaining task vector, with $\texttt{input_dim} = 2 + 2K + K$. For L4V, the policy network is an MLP that takes the AAV position, user relative positions, and the remaining task vector, with input_dim $= 2 + 2K + K$. The network uses fully connected layers of sizes 64, 64, and 32. Two output heads produce the control vector $\mathbf{u}[t] = [v[t], \theta[t]]^T$: the speed head applies a sigmoid activation scaled by $v_{\max}$ to enforce $v[t] \in [0, v_{\max}]$, and the heading head outputs $\theta[t]$ directly. The per-step objective is the sum of remaining tasks plus 0.01 times the sum of AAV–user distances, with an additional angle regularization weighted by 10^{-3}. A trial finishes when the total remaining task is below $10^{-3}K$, and training stops early when the objective change is below 10^{-3} for five consecutive iterations. The AAV maximum speed is 0.2, the horizon length is at most 500 steps, and the training budget is at most 2000 iterations.

B. Baseline Methods

To evaluate the performance of the proposed L4V framework comprehensively, we compare it against four representative algorithms covering value-based reinforcement learning, actor-critic methods, generative models, and evolutionary heuristics.

- **Genetic Algorithm (GA)** [39]: This method represents the class of evolutionary computation and meta-heuristic optimization. GA maintains a population of candidate trajectories and iteratively improves them through selection, crossover, and mutation operations. We employ GA to demonstrate the performance of derivative-free optimization approaches which do not rely on gradient information but typically require substantial computational resources for convergence.
- **Deep Q-Network (DQN)** [40]: This algorithm is a prominent value-based reinforcement learning method. Since DQN is inherently designed for discrete action spaces, we discretize the continuous heading angle of the AAV into a finite set of possible directions. The inclusion of DQN highlights the challenges associated with applying discrete control policies to smooth physical trajectory planning tasks.
- **Advantage Actor-Critic (A2C)** [41]: We utilize A2C as a representative on-policy actor-critic algorithm. A2C optimizes a stochastic policy by estimating the advantage function. This baseline serves to illustrate the impact of high variance in stochastic gradient estimation compared to the deterministic adjoint gradients used in our proposed method.
- **Deep Deterministic Policy Gradient (DDPG)** [36]: This algorithm is a state-of-the-art off-policy actor-critic method specifically designing for continuous control problems. DDPG learns a deterministic policy using the Bellman equation and replay buffers. It serves as the primary reinforcement learning benchmark to demonstrate the superiority of physics-informed supervision over scalar reward signals in complex environments.

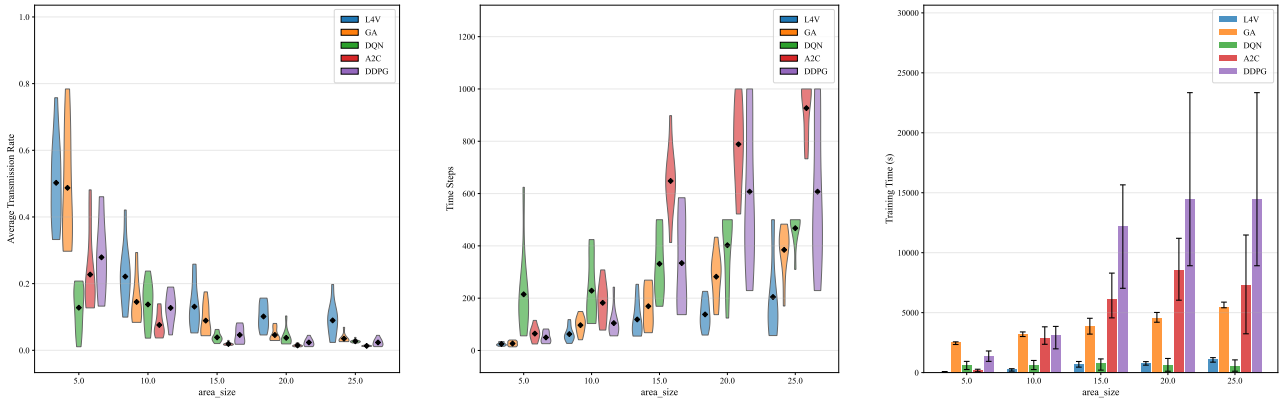
C. Impact of Environmental Scale

We further investigate the scalability of the proposed L4V framework by varying the dimensions of the mission area to evaluate robustness when the state space expands and the task horizon lengthens. Fig. 2 illustrates the comparative performance metrics across increasing area sizes. Specifically, Fig. 2(c) presents the computational time required for each method to reach convergence. The L4V framework maintains a consistently low training overhead regardless of the environment size. This efficiency stems from the deterministic nature of the adjoint gradient computation which provides precise action-level supervision without requiring exhaustive exploration. In contrast, the training times for reinforcement learning baselines such as A2C and DDPG increase drastically as the area expands because the sparse reward problem becomes more severe in larger environments.

In terms of mission efficiency, Fig. 2(b) depicts the distribution of time steps required to complete the data collection mission. The proposed L4V method consistently achieves the lowest median completion time with minimal variance across all tested scales. This indicates that L4V effectively learns to navigate the larger spatial domain to serve users promptly while baselines exhibit substantial performance degradation. Furthermore, Fig. 2(a) analyzes the average transmission rate achieved during the mission. While the data rate naturally decreases for all methods as the area size increases due to longer average communication distances, L4V consistently outperforms the baselines with a distribution concentrated towards higher values. This superiority demonstrates that the variational optimization approach successfully captures the geometric coupling between AAV positioning and channel quality even in challenging large-scale scenarios.

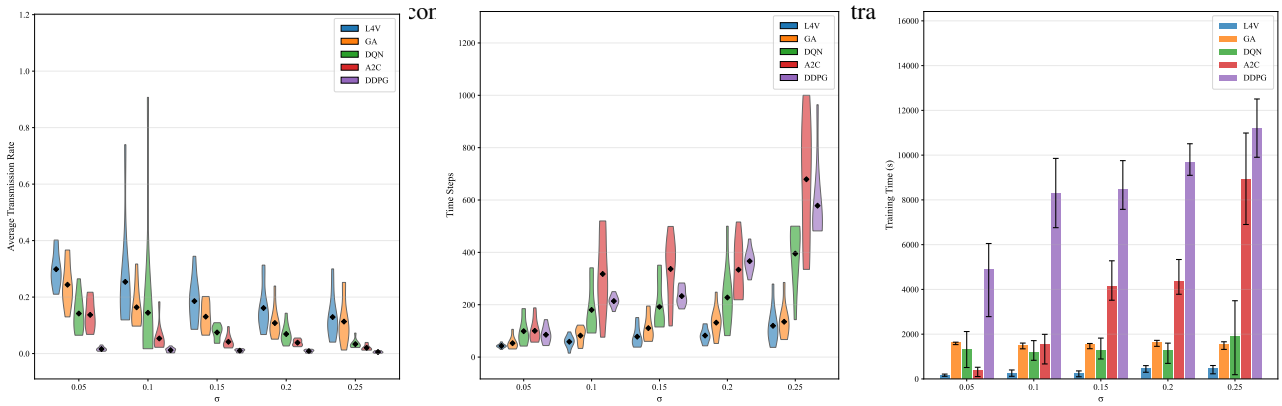
D. Impact of Noise Power

We evaluate the robustness of the proposed L4V framework against environmental stochasticity by varying the noise power parameter σ. This analysis is critical for understanding how well the learned policies adapt to channel degradation and signal uncertainty. Fig. 3 summarizes the performance metrics as the noise level increases from 0.05 to 0.25.Fig. 3(c) illustrates the training time required to reach convergence. The L4V method exhibits remarkable stability with negligible increase in computational cost even as the environment becomes noisier. This resilience is attributed to the utilization of deterministic adjoint gradients which provide consistent descent directions regardless of the stochastic reward noise. In contrast, reinforcement learning baselines such as A2C and DDPG experience a significant surge in training time at higher noise levels. The



(a) Average transmission rate distribution. (b) Mission completion time steps distribution. (c) Training time overhead to convergence.

Fig. 2: Performance scalability analysis under varying environmental scales. The proposed L4V framework is compared against baselines in tern cor tra



(a) Average transmission rate performance. (b) Mission completion time steps distribution. (c) Training time overhead to convergence.

Fig. 3: Performance robustness analysis under varying noise power levels σ. The proposed L4V framework is compared against baselines in terms of (a) communication quality, (b) task completion efficiency, and (c) computational training cost.

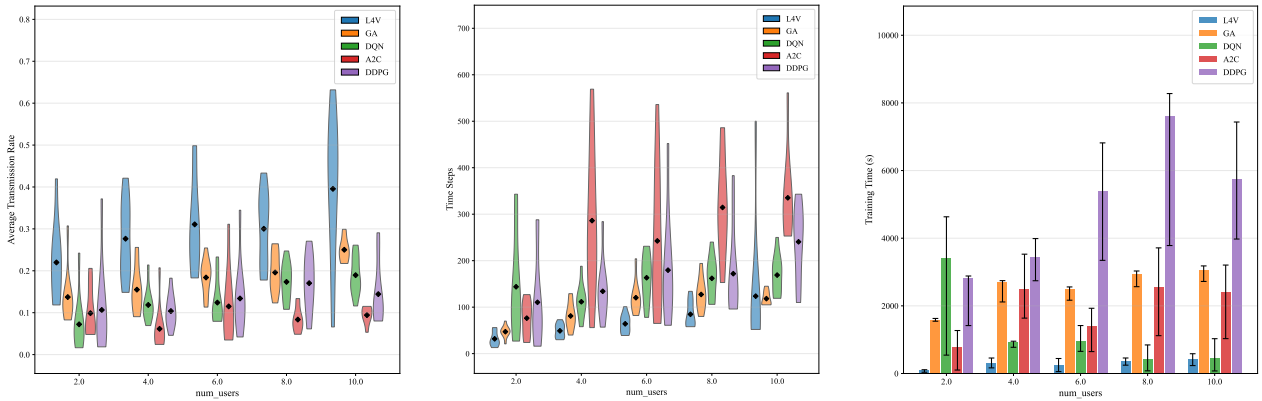
high variance in the reward signal under these conditions exacerbates the difficulty of credit assignment for standard RL agents, necessitating substantially more interaction steps to stabilize the policy.The impact of noise on mission efficiency is depicted in Fig. 3(b). L4V consistently achieves the lowest median completion time with tight variance bounds across the entire range of σ. This indicates that the variational policy effectively navigates the trade-off between flight time and data collection even when channel conditions are poor. Conversely, the baselines show a marked degradation in performance, with DDPG and A2C requiring significantly longer times to complete the task as σ increases. Finally, Fig. 3(a) confirms the superiority of L4V in terms of communication quality. Although the average transmission rate naturally declines for all methods with increasing noise, L4V maintains a higher throughput compared to the baselines. This suggests that the L4V agent learns to optimize the AAV trajectory to exploit favorable geometric positions that mitigate the effects of high noise power.

E. Impact of User Density

We investigate the scalability of the proposed framework by varying the number of ground users to assess performance as the task complexity increases. This experiment is crucial for validating the ability of the AAV to efficiently schedule and serve multiple nodes in a dense network. Fig. 4 summarizes the performance comparisons across different user densities.

Fig. 4(c) demonstrates the computational efficiency of the algorithms. The L4V framework maintains a consistently low and stable training time even as the number of users increases. This scalability stems from the fact that the adjoint method computes exact gradients by backpropagating through the system dynamics, effectively decoupling the complexity of the gradient computation from the combinatorial nature of the exploration space. In stark contrast, reinforcement learning baselines such as DDPG and A2C exhibit a dramatic increase in training overhead. As the state space grows with the number of users, these model-free agents require exponentially more samples to resolve the credit assignment problem and converge to a viable policy.

The mission execution efficiency is analyzed in Fig. 4(b). L4V consistently achieves the lowest median completion time with minimal variance, indicating that the learned policy effectively solves the underlying combinatorial routing problem inherent in multi-user data collection. The baselines, particularly DDPG and A2C, show high variance and significantly longer mission durations, suggesting that they often converge to suboptimal local minima where the AAV takes inefficient paths between users. Furthermore, Fig. 4(a) shows the average transmission rate. L4V maintains superior throughput perfor-



(a) Average transmission rate performance. (b) Mission completion time steps distribution. (c) Training time overhead to convergence.

Fig. 4: Performance scalability analysis with respect to user density. The proposed L4V framework is evaluated against baselines as the number of users increases in terms of (a) communication quality, (b) task completion efficiency, and (c) computational training cost.

mance compared to the baselines across all density settings. This indicates that the variational approach not only optimizes the flight path for speed but also successfully maximizes the spectral efficiency by positioning the AAV in favorable geometric locations relative to the active users.

VII. Conclusion

In this paper, we have proposed "Learn for Variation" (L4V), a novel trajectory optimization framework that integrates variational optimal control theory with end-to-end differentiable simulation. By exploiting the reversibility of the computational graph, we have established a direct supervision mechanism that utilizes exact adjoint gradients to train neural policies, effectively overcoming the sample inefficiency and high variance limitations of reward-driven reinforcement learning. Simulation results demonstrate that the proposed approach significantly enhances the operational agility of practical AAV networks, achieving superior convergence speed and robust task completion rates compared to state-of-the-art baselines even in large-scale, stochastic environments. Future work will focus on extending this differentiable paradigm to decentralized multi-agent cooperative scenarios and validating the learned policies on hardware-in-the-loop experimental platforms.

References

[1] X. Shen, J. Gao, M. Li, C. Zhou, S. Hu, M. He, and W. Zhuang, "Toward immersive communications in 6g," *Frontiers in Computer Science*, vol. 4, p. 1068478, 2023.

[2] N. Cheng, F. Chen, W. Chen, Z. Cheng, Q. Yang, C. Li, and X. Shen, "6G omni-scenario on-demand services provisioning: vision, technology and prospect(in chinese)," *Sci Sin Inform*, vol. 54, pp. 1025–1054,, 2024.

[3] X. Wang, L. Fu, N. Cheng, R. Sun, T. Luan, W. Quan, and K. Aldubaikhy, "Joint flying relay location and routing optimization for 6g uav–iot networks: A graph neural network-based approach," *Remote Sensing*, vol. 14, no. 17, p. 4377, 2022.

[4] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Select. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, 2019.

[5] X. Wang, K. Tao, N. Cheng, Z. Yin, Z. Li, Y. Zhang, and X. Shen, "Radiodiff: An effective generative diffusion model for sampling-free dynamic radio map construction," *IEEE Transactions on Cognitive Communications and Networking*, vol. 11, no. 2, pp. 738–750, 2025.

[6] X. Wang, Q. Zhang, N. Cheng, R. Sun, Z. Li, S. Cui, and X. Shen, "Radiodiff-k2: Helmholtz equation informed generative diffusion model for multi-path aware radio map construction," *IEEE Journal on Selected Areas in Communications*, 2025.

[7] Y. Zhang, D. J. Love, J. V. Krogmeier, C. R. Anderson, R. W. Heath, and D. R. Buckmaster, "Challenges and opportunities of future rural wireless communications," *IEEE Communications Magazine*, vol. 59, no. 12, pp. 16–22, 2022.

[8] J. Lyu, Y. Zeng, and R. Zhang, "Uav-aided offloading for cellular hotspot," *IEEE Transactions on Wireless Communications*, vol. 17, no. 6, pp. 3988–4001, 2018.

[9] L. Gupta, R. Jain, and G. Vaszkun, "Survey of important issues in uav communication networks," *IEEE communications surveys & tutorials*, vol. 18, no. 2, pp. 1123–1152, 2015.

[10] R. Ding, F. Gao, and X. S. Shen, "3d uav trajectory design and frequency band allocation for energy-efficient and fair communication: A deep reinforcement learning approach," *IEEE Transactions on Wireless Communications*, vol. 19, no. 12, pp. 7796–7809, 2020.

[11] Y. Zeng and R. Zhang, "Energy-efficient uav communication with trajectory optimization," *IEEE Transactions on wireless communications*, vol. 16, no. 6, pp. 3747–3760, 2017.

[12] H. Zhao, H. Wang, W. Wu, and J. Wei, "Deployment algorithms for uav airborne networks toward on-demand coverage," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 9, pp. 2015–2031, 2018.

[13] Y. Huo, X. Dong, T. Lu, W. Xu, and M. Yuen, "Distributed and multilayer uav networks for next-generation wireless communication and power transfer: A feasibility study," *IEEE Internet of Things Journal*, vol. 6, no. 4, pp. 7103–7115, 2019.

[14] N. Cheng, X. Wang, Z. Li, Z. Yin, T. H. Luan, and X. Shen, "Toward enhanced reinforcement learning-based resource management via digital twin: Opportunities, applications, and challenges," *IEEE Network*, vol. 39, no. 1, pp. 189–196, 2025.

[15] P. Wang, X. Luo, Z. Zhou, Y. Zang, and L. Hu, "Key technology for remote sensing information acquisitionbased on micro uav," *Transactions of the Chinese Society of Agricultural Engineering*, vol. 30, no. 18, pp. 1–12, 2014.

[16] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 2006.

[17] R. Zhou, X. Wu, H. Tan, and R. Zhang, "Two time-scale joint service caching and task offloading for uav-assisted mobile edge computing," in *IEEE INFOCOM 2022-IEEE conference on computer communications*. IEEE, 2022, pp. 1189–1198.

[18] Y. Zeng, J. Chen, J. Xu, D. Wu, X. Xu, S. Jin, X. Gao, D. Gesbert, S. Cui, and R. Zhang, "A tutorial on environment-aware communications via channel knowledge map for 6g," *IEEE communications surveys & tutorials*, vol. 26, no. 3, pp. 1478–1519, 2024.

[19] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: Opportunities and challenges," *IEEE Communications magazine*, vol. 54, no. 5, pp. 36–42, 2016.

[20] C. H. Liu, X. Ma, X. Gao, and J. Tang, "Distributed energy-efficient multi-uav navigation for long-term communication coverage by deep reinforcement learning," *IEEE Transactions on Mobile Computing*, vol. 19, no. 6, pp. 1274–1285, 2019.

[21] Y. Mao, J. Zhang, and K. B. Letaief, "Joint task offloading scheduling and transmit power allocation for mobile-edge computing systems," in *2017 IEEE wireless communications and networking conference (WCNC)*. IEEE, 2017, pp. 1–6.

[22] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE signal processing magazine*, vol. 34, no. 6, pp. 26–38, 2017.

[23] C. H. Liu, Z. Chen, J. Tang, J. Xu, and C. Piao, "Energy-efficient uav control for effective and fair communication coverage: A deep reinforcement learning approach," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 9, pp. 2059–2070, 2018.

[24] E. Heiden, D. Millard, E. Coumans, Y. Sheng, and G. S. Sukhatme, "Neuralsim: Augmenting differentiable simulators with neural networks," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 9474–9481.

[25] A. Ioffe, "Euler-lagrange and hamiltonian formalisms in dynamic optimization," *Transactions of the american mathematical society*, vol. 349, no. 7, pp. 2871–2900, 1997.

[26] N. Kim, S. Cha, and H. Peng, "Optimal control of hybrid electric vehicles based on pontryagin's minimum principle," *IEEE Transactions on control systems technology*, vol. 19, no. 5, pp. 1279–1287, 2010.

[27] P. Tseng, "Convergence of a block coordinate descent method for nondifferentiable minimization," *Journal of optimization theory and applications*, vol. 109, no. 3, pp. 475–494, 2001.

[28] A. Liu, V. K. Lau, and B. Kananian, "Stochastic successive convex approximation for non-convex constrained stochastic optimization," *IEEE Transactions on Signal Processing*, vol. 67, no. 16, pp. 4189–4203, 2019.

[29] Y. Yang, M. Pesavento, Z.-Q. Luo, and B. Ottersten, "Inexact block coordinate descent algorithms for nonsmooth nonconvex optimization," *IEEE Transactions on Signal Processing*, vol. 68, pp. 947–961, 2019.

[30] Z.-q. Luo, W.-k. Ma, A. M.-c. So, Y. Ye, and S. Zhang, "Semidefinite relaxation of quadratic optimization problems," *IEEE Signal Processing Magazine*, vol. 27, no. 3, pp. 20–34, 2010.

[31] S. El Khediri, W. Fakhet, T. Moulahi, R. Khan, A. Thaljaoui, and A. Kachouri, "Improved node localization using k-means clustering for wireless sensor networks," *Computer Science Review*, vol. 37, p. 100284, 2020.

[32] Y. Xu and C. Che, "A brief review of the intelligent algorithm for traveling salesman problem in uav route planning," in *2019 IEEE 9th international conference on electronics information and emergency communication (ICEIEC)*. IEEE, 2019, pp. 1–7.

[33] D. Rojas Viloria, E. L. Solano-Charris, A. Muñoz-Villamizar, and J. R. Montoya-Torres, "Unmanned aerial vehicles/drones in vehicle routing problems: a literature review," *International Transactions in Operational Research*, vol. 28, no. 4, pp. 1626–1657, 2021.

[34] D. Erdos, A. Erdos, and S. E. Watkins, "An experimental uav system for search and rescue challenge," *IEEE Aerospace and Electronic Systems Magazine*, vol. 28, no. 5, pp. 32–37, 2013.

[35] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[36] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *International conference on machine learning*. Pmlr, 2014, pp. 387–395.

[37] S. Dankwa and W. Zheng, "Twin-delayed ddpg: A deep reinforcement learning technique to model a continuous movement of an intelligent robot agent," in *Proceedings of the 3rd international conference on vision, image and signal processing*, 2019, pp. 1–5.

[38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[39] A. Lambora, K. Gupta, and K. Chopra, "Genetic algorithm-a literature review," in *2019 international conference on machine learning, big data, cloud and parallel computing (COMITCon)*. IEEE, 2019, pp. 380–384.

[40] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[41] I. Grondman, L. Busoniu, G. A. Lopes, and R. Babuska, "A survey of actor-critic reinforcement learning: Standard and natural policy gradients," *IEEE Transactions on Systems, Man, and Cybernetics, part C (applications and reviews)*, vol. 42, no. 6, pp. 1291–1307, 2012.